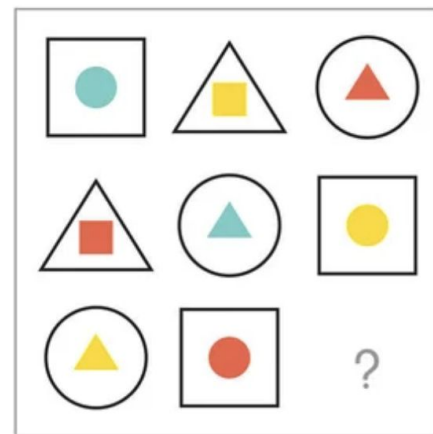
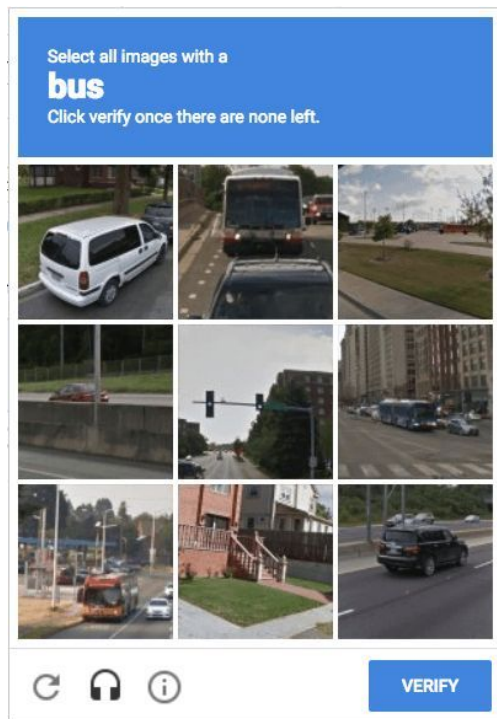
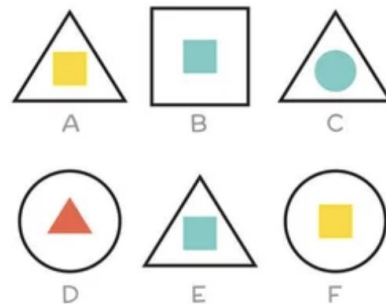


HA1 Übung Angew Prog

Lehrbeauftragter: Long Nguyen



CHOOSE ANSWER



Thema: Testen

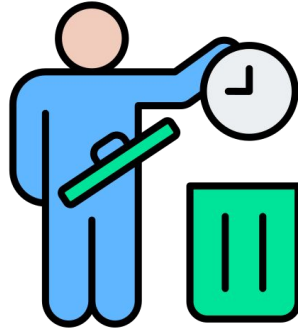
3 Typen von Entwicklern

- 1. Typ: Kein Testen

3 Typen von Entwicklern

- 1. Typ: Kein Testen

Testen = “Zeitverschwendung”



3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen

3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen

“Falls der Warenkorbwert 50 oder mehr ist, gewähre einen Rabatt von 10 %.”

```
public double calculateTotal(double cartValue) {  
    if (cartValue > 50.0) {  
        return cartValue * 0.9;  
    }  
    return cartValue;  
}
```

3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen

“Falls der Warenkorbwert 50 oder mehr ist, gewähre einen Rabatt von 10 %.”

```
public double calculateTotal(double cartValue) {  
    if (cartValue > 50.0) {  
        return cartValue * 0.9;  
    }  
    return cartValue;  
}
```

Fehler: Fall Warenkorbwert = 50 vergessen

3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen

“Falls der Warenkorbwert 50 oder mehr ist, gewähre einen Rabatt von 10 %.”

```
@Test
public void testCalculateTotal_ConfirmationBias() {
    assertEquals(90.0, calculateTotal(100.0));

    assertEquals(20.0, calculateTotal(20.0));
}
```

Fehler: Fall Warenkorbwert = 50 vergessen

3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen
- 3. Typ: Testen, dann Coden

3 Typen von Entwicklern

- 1. Typ: Kein Testen
- 2. Typ: Coden, dann Testen
- 3. Typ: Testen, dann Coden

“Falls der Warenkorbwert 50 oder mehr ist, gewähre einen Rabatt von 10 %.”

```
@Test
public void testCalculateTotal() {
    assertEquals(49.0, calculateTotal(49.0));
    assertEquals(45.0, calculateTotal(50.0));
    assertEquals(45.9, calculateTotal(51.0));
}
```

Assert Funktionen

- `assertEquals(expected, actual) ↔ assertEquals(unexpected, actual)`

Assert Funktionen

- `assertEquals(expected, actual) ↔ assertEqualsUnexpected(unexpected, actual)`
- `assertTrue(condition) ↔ assertFalse(condition)`

Assert Funktionen

- assertEquals(expected, actual) ↔ assertEquals(unexpected, actual)
- assertTrue(condition) ↔ assertFalse(condition)
- assertNotNull(actual) ↔ assertNull(actual)

Assert Funktionen

- assertEquals(expected, actual) ↔ assertNotEquals(unexpected, actual)
- assertTrue(condition) ↔ assertFalse(condition)
- assertNotNull(actual) ↔ assertNull(actual)
- assertThrows(ExpectedException, Executable)

